ML System Design in a Hurry

Early Access

Delivery Framework

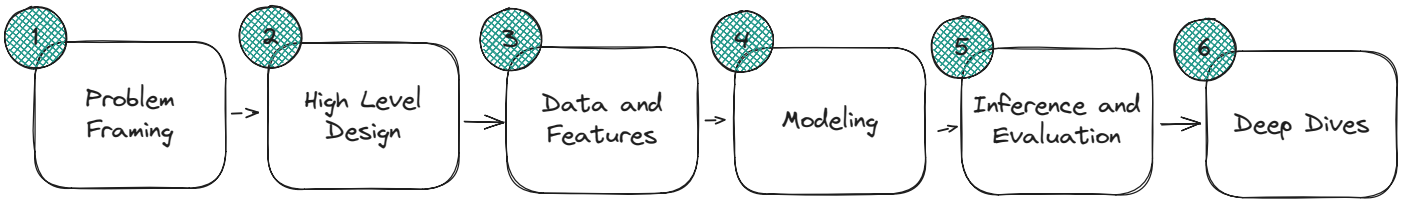
The best way to structure your system design interviews to structure your thoughts and focus on the most important aspects, built by FAANG managers and staff engineers.

ML design interviews can be daunting, each decision you make uncovers many more questions that might need your attention. There's a constant tradeoff between breadth and depth. And worse, across the industry there is substantially less standardization (than SWE System Design) in how the interview is expected to flow.

Fortunately, the goal of the interview is simple: to give your interviewer a clear picture of how effectively you can solve real-world ML problems. And your job is to show this by taking an ambiguous problem and clearly demonstrating how you'd approach it—not just from a technical standpoint, but in a way that solves a real business need and drives meaningful impact for the organization.

Our delivery framework offers a structure for your interview: a series of steps and timings we recommend you follow to ensure you're able to get to the load-bearing bits of the interview. Think of them like guideposts. They're not hard rules and if your interviewer drags you off course you should follow their lead. But left to your own by following them you'll make sure you leave the interview covering the most important bits that are evaluated across several major companies.

Here's the delivery framework:



ML System Design Interview Structure

Let's walk through each section of the framework, highlighting what you should be doing and what the interviewer will be looking for.

Problem Framing (5-7 minutes)

In ML design interviews, your interviewer will typically start the interview with a high-level problem to solve with a lot of ambiguity. Your job at the outset is to do three things: 1) clarify the problem and make sure you understand it, 2) establish a high-level business objective for the problem, and 3) turn that into an ML objective you can build around. Let's talk about each briefly.

Clarify the Problem

Start by asking targeted questions to understand the scope and constraints of the problem. You want to understand who the users are, what their pain points are, and what the current solutions look like if any exist. It's also important to understand the scale requirements, such as the number of users or requests per second and whether inference must be real-time or batch, as well as any specific constraints like latency or privacy concerns.

For example, if you're asked to design a recommendation system for an e-commerce platform, you might ask about the size of the product catalog, the number of daily active users, where the recommendations will be shown, what the latency requirements are, and whether there exists a current system in place.

> One of the most common mistakes we see from candidates is jumping straight into the problem without taking time to clarify the scope and constraints. Even if the interviewer has given you a problem statement, take time to ask targeted questions to truly understand the problem before moving forward. Not only are interviewers evaluating you on your ability to interrogate the problem, but you'll also avoid a painful situation of having to backtrack later as requirements clarify.

Interviewers are typically not "grading" this section, but strong candidates can set themselves apart by digging quickly into what makes a problem interesting or challenging. A typical staff-level candidate will immediately recognize the core challenge and start probing around the things that teams will be working on for the next few years.

Establish a Business Objective

After we've got a clearer view of the problem we need to establish a business objective.

In building real ML systems, the ultimate objective for the solution is usually not the loss function of the model. As an example, consider building a system to detect sensitive content on Facebook. While the problem is clearly asking you to build some form of classifier, the intent of that classifier might be to eliminate legal risks, or reduce unwanted exposures to users. These aren't the same thing!

In this section, we want to articulate a clear business objective that your ML system will help achieve. This might be increasing user engagement, reducing operational costs, improving user satisfaction, mitigating risks, or generating revenue. The key is to be **specific** about what success looks like from a business perspective.

Make special note here of places where your business objective and a naive ML objective might differ. If we want to reduce unwanted exposure of harmful content to users, for example, the posts which get views (especially lots of views) are infinitely more important than those which get none!

> Be specific about the business objective. "Improve user experience" or "increase revenue" is often too vague to be actionable. "Increase click-through rate on recommendations" is a much clearer objective which can guide downstream decisions.
>
> You don't need to be precise (no bonus points for +10% revenue, especially if the current revenue is not clear), but you should be able to articulate what success looks like directionally.

In most ML teams, and especially in big companies, teams will spend years working on optimizing a fairly narrow business objective. ML engineers on these teams understand these objectives inside and out as they look for ways to optimize them. Interviewers are trying to get a sense for how effective you are in this pursuit.

Decide on an ML Objective

Once you have a clear business objective, you need to translate it into a concrete ML objective. This involves determining what type of ML task(s) you're dealing with (classification, regression, ranking, clustering, etc.), defining what success looks like in ML terms, and identifying the key metrics you'll use to evaluate your model.

For our e-commerce recommendation system, the ML objective might be to build a ranking model that predicts the probability of a user purchasing a product given their browsing history and other contextual information. The primary metric might be precision@k (the proportion of recommended items that are actually purchased) or normalized discounted cumulative gain (NDCG) to account for the order of recommendations.

> Don't get too hung up on the hyperparameters of your loss function. You rarely need to decide on a k or determine at which precision level you'll evaluate recall and the discussions around them are too vague to give your interviewer signal on your ability to build a good system.

Deciding on an ML objective essentially sets the stage for the rest of the interview. Not only are interviewers looking for you to build clarity here, but you'll also be using this objective to guide the rest of your work.

Summary

Green Flags

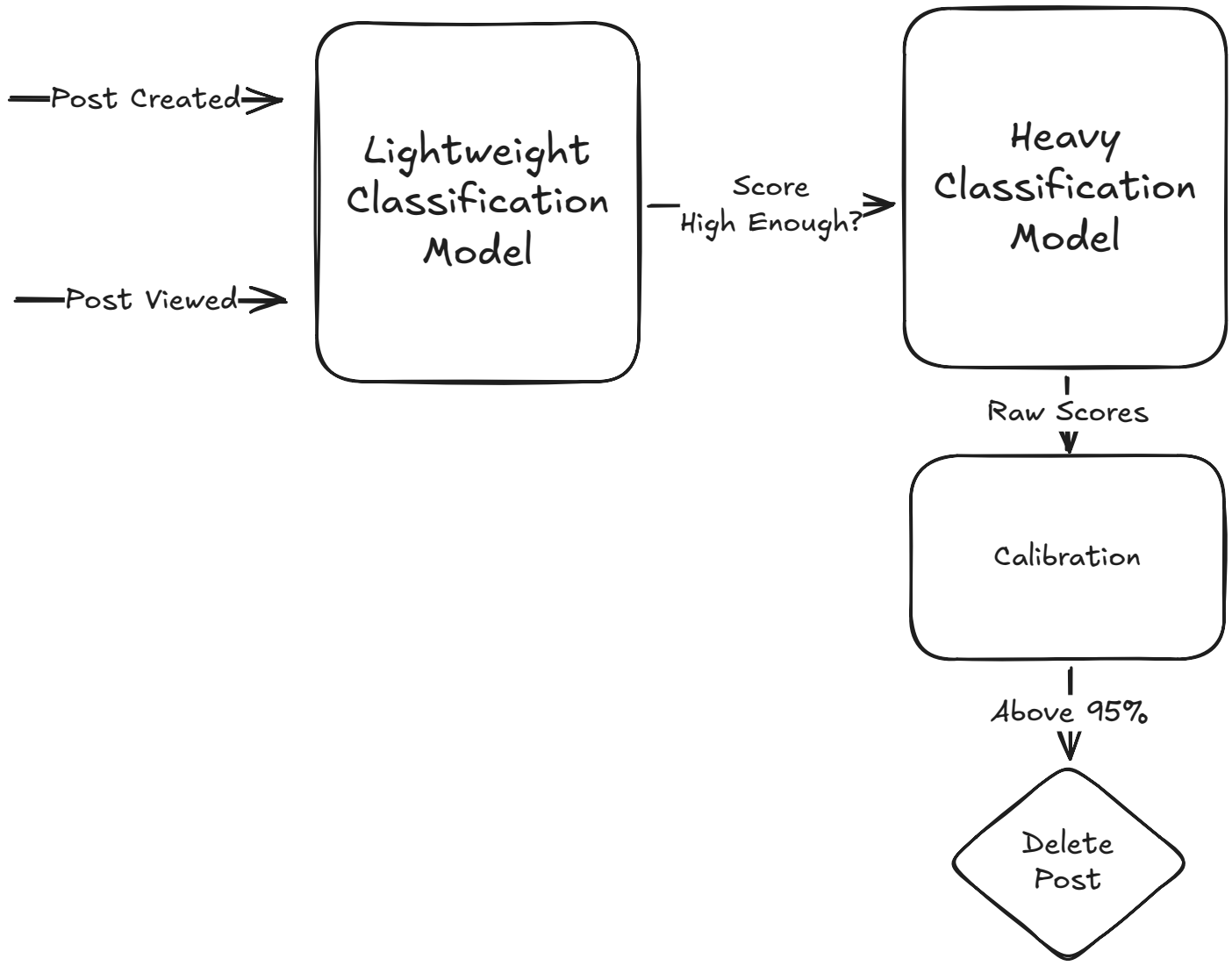
- You've asked detailed questions which get at the heart of the problem
- You established a clear business objective which gives you a basis to optimize
- You've articulated a clear ML objective which will guide the rest of your work

Red Flags

- You assumed a naive ML-focused objective is sufficient
- Your questions didn't uncover what makes the problem interesting or challenging
- Your ML objective doesn't give you sufficient clarity for the rest of your design

High-level Design (2-3 minutes)

Once we have the problem clarified and objectives in place, we can set up a scaffolding of how the pieces will fit together. This is usually a simple block diagram showing the inputs and outputs of your system together with the key components in between, but could be as light-weight as a verbal description.



Sample High-Level Design for a Content Moderation System

Don't get hung up on the diagram itself in the interview, as it's a mechanism for communicating your ideas — not a final product on which you'll be assessed. Sometimes candidates will spend a lot of time trying to perfect the diagram, which only distracts from the real signal your interviewer is looking for.

For applied ML design interviews, you can take for granted that there will be high performance feature extraction and model inference components. Make sure your design includes the entire

lifecycle from data inputs through to the actions you might take. There's frequently some interesting twists and nuances that become more obvious only when you walk the entire lifecycle.

The purpose of this section is to provide clarity on how the rest of our modelling discussion is situated, but be wary of going too deep into SWE-level technical details like database choices or API design. While these topics are important for ML infra interviews, for applied ML interviews they're rarely the focus as larger companies tend to have established ML infra teams who will handle these details.

Data and Features (10 minutes)

With clarity about the problem and ML objectives in place, we can now think about the data we'll use to train our model.

We'll go through this section sequentially: from the raw data we have available, to the features we'll use, to how they are represented to be used by the model. That said, don't worry if you need to jump back and forth between steps — it's completely normal to have new ideas or insights as you go.

Training Data

First, discuss the data sources you'll need to train your model. Consider what existing data you have access to and whether you need to collect new data. If you don't know whether you have something available, just ask!

Think about how much data you'll need and potential biases or quality issues in the data. If you need labeled data, discuss how you will handle the labeling process, including the use of direct labels or proxy signals such as clicks or user interactions.

> It can be helpful to think about data in buckets: supervised, semi-supervised, and unsupervised. Most candidates will readily acknowledge the supervised data bucket, but great solutions often involve leveraging the order(s) of magnitude more available data in the other buckets.

For our e-commerce recommendation system, you might use historical purchase data, browsing behavior, product metadata, user profiles, and possibly external data like seasonal trends or competitor pricing. You'd want to consider how to handle cold-start problems for new users or products, and how to ensure your training data doesn't reinforce existing biases in purchasing patterns.

> Don't assume perfect data availability. In real-world ML problems, data collection and preparation often consume the majority of development time.

Features

Next, identify the features that will be most predictive for your model. Start with the raw data fields available, then consider what transformations or aggregations might be useful. Think about temporal aspects, such as user behavior over time, and how domain knowledge can inform your feature selection.

Be careful of getting caught up in a naive feature dump! Some candidates will spend this section rattling off as many features as they can think of. For many problems, there's a nearly infinite number of potential features you can use and this doesn't show your interviewer any insight and burns time for other discussions.

For our recommendation system, features might include user demographics, past purchase history, browsing patterns, product categories, price points, and contextual information like time of day or device type. You might also create derived features like the similarity between products based on co-purchasing patterns or the recency and frequency of user interactions with certain product categories. Some of these features might be online and queried directly (meaning they are up-to-date) while others might be computed offline or in batches and stored in a feature store.

Discuss how you'll represent different types of data in your model. For categorical features, you might use one-hot encoding or embeddings. For text data, you could use bag-of-words, word embeddings, or transformer models. Images or other unstructured data require their own specialized representations. You'll also need to consider how to normalize numerical features and handle missing values.

In our e-commerce example, you might use embeddings to represent products and users in a shared latent space, one-hot encoding for categorical features like product categories, and normalized values for numerical features like prices or ratings. For product descriptions or reviews, you might use pre-trained language models to extract semantic information.

> Prioritize features based on their expected predictive power and implementation feasibility. Not all theoretically useful features are practical to implement or worth the development and operational costs.

Summary

By the end of our data and features discussion, we should have a clear understanding of the data we have available, the features we'll use, and how they'll be represented to be used by the model.

Green Flags

- You've creatively used not just supervised data, but semi-supervised and unsupervised data
- Your features are impactful and have a solid hypothesis for why they'll be predictive
- You've discussed encoding and representation concerns where appropriate

Red Flags

- You've dumped a laundry list of features, many of which aren't impactful or practical
- You've left ambiguous how your features will be represented or used by the model

Modeling (10 minutes)

With a clear understanding of the problem, objectives, and data, you can now discuss your modeling approach.

Benchmark Models

While many junior candidates will immediately dive into the latest and greatest models, experienced candidates will start with simple models that could serve as baselines. These might be heuristic approaches, simple statistical models, or basic machine learning algorithms. Establishing baselines helps you understand the problem better and provides a reference point for evaluating more complex models.

For our recommendation system, a simple baseline might be popularity-based recommendations (showing the most popular products) or basic collaborative filtering using user-item interaction matrices. These approaches are easy to implement and often perform surprisingly well, making them good starting points.

The point here isn't to go into excessive detail on the baseline models, but propose a simple model which would give you a yardstick to compare against as you add complexity. This quickly moves the conversation from a theoretical one to a more grounded, practical one of tradeoffs: what are the costs and benefits of each incremental piece of complexity we add?

Baseline models are also gap fillers in the case you need to have multiple models to solve your problem. If you need candidate generators and rankers, for example, you can describe a simple candidate generator model so you have a complete system if you need to spend more time discussing the ranker.

Model Selection

With a baseline(s) in place, you can now discuss the models you would consider for this problem beyond the baseline. Talk through what model families are appropriate for this type of problem and the tradeoffs (cost, complexity, latency, interpretability, predictive power, etc.) between different model choices. Discuss how these models align with your constraints, such as latency requirements or interpretability needs.

For most problems, there is often a choice between a simpler, "classical" model and a deep learning model. Talking through both options shows that you understand the tradeoffs between different model families and can make an informed decision based on the problem at hand. It's not always the case that a deep learning model is the best approach (!), but in most cases it needs to be considered.

For applied ML roles, interviewers want to see a mix of theory and practice. The best approaches are generally in the family of ideas which have been tried and tested in the last 2-3 years. Don't neglect non-parametric approaches like ANN models which can be very effective on their own or complementary to other models for many problems.

Surveying recent citations from the public ML blogs of top companies will rarely leak the state of the art, but is often a passable baseline for interview purposes! Approaches that continue to stand the test of time are often ideal for production systems.

Interviewers want to see some breadth here. If the only option you have is a paper you read from NeurIPS this year, it's going to be hard to convince them that the proposed approach is robust enough for production. On the other hand, if you're missing out on research from the last 5-7 years, you run the risk that your interviewer will see your knowledge as dated.

Once you've had a decent discussion about options and tradeoffs, you'll want to make a choice to spend your time elaborating on **one** of the models your proposing: there simply isn't enough time to cover them all.

Model Architecture

With a proposed model in place, it's time to discuss its architecture in more detail. This includes the key components of the model, the key layers or parameters you would expect to need, the activation functions you would use, how you would handle regularization to prevent overfitting, and the loss function.

If you're proposing a deep learning approach for our recommendation system, you might describe a two-tower architecture with separate embeddings for users and items, followed by several fully connected layers with ReLU activations, and a final sigmoid output layer for predicting purchase probability. You would also discuss regularization techniques like dropout or L2 regularization to prevent overfitting.

This is the point for you to thread the needle between "I'm going to use a deep learning model" (too high-level!) and "I'll have 4 fully-connected layers with 1024 neurons each" (this is something you can only establish empirically!).

Interviewers are using this as a bullshit test: do you really understand the model you're proposing? Does it seem like you've built something similar in practice? Expect follow-up questions here and be prepared to confidently defend your choices.

> It's ok to volunteer that you don't know some details. "I've worked mostly on classification systems so I'm not intimately familiar with the details of ranking models, but here's what I know" is a great start. Interviewers are expecting some degree of unfamiliarity given the breadth of ML research and the point of the interview.
>
> That said, in those places where you don't know the answer the onus will be on you to showcase your ability to generalize the knowledge you do have to the new domain. The critical question of the interview is "can we take this engineer's knowledge and use it effectively in a new space?" — the more cleanly you can help your interviewer see that, the better you'll do.

Green Flags

- You've established a simple and fast baseline to compare against a more complex approach you proposed
- You described a few different approaches, their tradeoffs, and which you'd prefer for the problem at hand
- You've given sufficient detail of the model architecture to both explain

Red Flags

- You jump straight to a complex, expensive model without considering simpler alternatives
- You've hand-waved the details of the model architecture such that the interviewer isn't confident you've built something similar in practice
-

Inference and Evaluation (7 minutes)

Now that you've designed your model, you need to discuss how it will be deployed and evaluated.

Evaluation Design and Metrics

Discuss how you'll evaluate your model both offline and online. Offline evaluation uses historical data to estimate how well your model will perform in production, while online evaluation measures the actual impact of your model on real users.

For offline evaluation of our recommendation system, you might use metrics like precision, recall, NDCG, or mean average precision on a held-out test set. For online evaluation, you would design A/B tests to measure the impact on business metrics like click-through rate, conversion rate, or average order value. Discussing how you'd measuring operational concerns like cost and latency can also be helpful to demonstrate the breadth of your experience.

Our **Evaluation Guide** has more details on how to approach evaluation design and metrics.

> Always tie your evaluation metrics back to the business objective. A model that performs well on ML metrics but doesn't improve business outcomes isn't valuable.

Inference Considerations

Many problems become more interesting when you try to operationalize them. Dealing with the scale, latency considerations, and costs of inference are all important parts of building a production-ready system. In this section we want to discuss practical considerations: do we need to distill the model? Provide caching? Is it reasonable to quantize the model, or prune it?

Depending on the question, inference-time considerations might be more or less important. If your inference can be done offline at a small scale, there may be not much to talk about here. On the other hand, if you're dealing with massive scale and sensitive latency requirements, expect the interviewer to push you to consider these aspects.

> Inference-time considerations are a great way to demonstrate practical depth of knowledge. Candidates who are focused on lab work in Jupyter notebooks are often not

> thinking through these aspects of the problem, which can be a deal-breaker for applied roles.

Summary

Green Flags

- You've established clear offline and online evaluation metrics that tie back to business objectives
- You've considered practical inference constraints like latency, cost, and scale
- You've proposed concrete ways to optimize inference (e.g. caching, quantization, pruning) where relevant

Red Flags

- Your evaluation metrics are disconnected from business objectives
- You've ignored practical inference considerations in favor of model accuracy alone
- You've proposed complex inference optimizations without justifying their necessity

Deep Dives (Remaining time)

In the final part of the interview, you'll have the opportunity to go deeper into specific aspects of your design. This might be driven by the interviewer's questions or by areas you've identified as particularly challenging or interesting. These will vary highly dependent on the problem you're solving but a few common categories are:

Handling Edge Cases

Discuss how your system handles edge cases like cold-start problems for new users or items, data sparsity, or seasonal trends. For our recommendation system, you might discuss techniques like content-based filtering for new items or exploration strategies like epsilon-

greedy for new users. Or for systems which are trained on non-representative data, how can you adjust the model to mitigate biases.

Scaling Considerations

Address how your system will scale as the user base or data volume grows. This might involve discussing distributed training, efficient serving architectures, or caching strategies.

Monitoring and Maintenance

Describe how you would monitor your model in production and when you would retrain it. Discuss what metrics you would track and what alerts you would set up. For our recommendation system, you might monitor metrics like click-through rate, diversity of recommendations, and model drift, and set up automated retraining when performance drops below a threshold.

> The deep dive section is your opportunity to demonstrate depth of knowledge in specific areas. Focus on the aspects most relevant to the problem and where you have the most expertise. That said, if the interviewer is driving, follow their lead! You won't be able to steer the interview to discuss your pet Reinforcement Learning topics if the interviewer is trying to get you to flesh out the details of your ranking model.

Summary

By following this delivery framework, you'll ensure that you cover all the key aspects of ML system design in a structured way, giving you space to show off your depth of knowledge.

Remember, the goal is not to design the perfect system in 45 minutes, but to demonstrate your thought process and ability to make reasonable trade-offs given the constraints.